

Logistic Regression, OvR & Performance Metrics

Complete Notes with Detailed Step-by-Step Explanation

(Har chhoti se chhoti concept ko asaan bhasha mein samjhaya gaya hai)

Is notes me kya-kya hai?

Logistic Regression • Sigmoid Function • Log Loss • Gradient Descent •
One-vs-Rest (OvR) multi-class • Confusion Matrix • Accuracy • Precision •
Recall • F-Beta Score

*Har topic ke saath teacher ke original handwritten notes ke pages bhi attach kiye
gaye hain, taaki explanation aur notes side-by-side dekh sako.*

Index

Part 1 — Logistic Regression	3
1. Introduction — Logistic Regression Kya Hai?	3
2. Linear Regression Ko Classification Ke Liye Kyun Nahi Use Kar Sakte?	4
3. Logistic Regression Kaise Solve Karta Hai — Sigmoid Function	7
4. Cost Function — Linear Regression Wala MSE Yahan Kyun Nahi Chalega?	9
5. Log Loss (Binary Cross-Entropy) — Logistic Regression Ka Cost Function	11
6. Gradient Descent — Convergence Algorithm	13
7. Logistic Regression for Multi-Class — One-vs-Rest (OvR)	16
Part 2 — Performance Metrics	21
8. Performance Metrics Kyun Zaroori Hain?	21
9. Confusion Matrix	22
10. Accuracy	23
11. Precision Aur Recall	25
12. Use Case 1 — Spam Classification (Precision)	26
13. Use Case 2 — Diabetes Prediction (Recall)	26
14. Assignment — Kal Stock Market Crash Hoga Ya Nahi?	28
15. F-Beta Score	28
16. Kab Konsa Metric Chunein?	32
17. Poore Notes Ka Quick Revision	32

Part 1 — Logistic Regression

Binary classification ka sabse pehla aur sabse important algorithm, uske baad multi-class ke liye One-vs-Rest (OvR).

1. Introduction — Logistic Regression Kya Hai?

Pichhle notes me humne **Simple Linear Regression** padhi thi, jisme hum ek **continuous (numeric) value** predict karte the — jaise height, price ya salary. Lekin real life me bahut saare problems aise bhi hote hain jahan answer koi number nahi, balki ek **category** hota hai — jaise *Pass ya Fail*, *Spam ya Not Spam*, *Diabetes hai ya nahi*. Aise problems ko **Classification** kehte hain.

Jab output ki sirf **do categories** hon, toh use **Binary Classification** kehte hain, aur iske liye sabse basic algorithm hai **Logistic Regression**.

Dhyaan dein: Naam me 'Regression' hai, lekin Logistic Regression ek **Classification** algorithm hai. Andar se yeh ek probability (0 se 1 ke beech ka number) nikalta hai, aur fir ek threshold (jaise 0.5) laga kar us probability ko class (0 ya 1) me badal deta hai.

Dataset Samajhna — Study Hours se Pass/Fail

Teacher ke example me dataset yeh hai. Input feature **Study Hours** hai aur output **Pass/Fail** — yaani binary categories. Computer text samajhta nahi hai, isliye hum **Fail = 0** aur **Pass = 1** encode kar dete hain.

Study Hours (X)	Result	Encoded Output (y)
2	Fail	0
3	Fail	0
4	Fail	0
5	Pass	1
6	Pass	1
7	Pass	1
12	Pass	1

Model ka kaam simple hai: **Study Hours** → [**Model**] → **PASS / FAIL**. Hum chahte hain ki naye student ke study hours dekh kar model bata sake ki woh pass hoga ya fail.

Linear Regression vs Logistic Regression — Ek Nazar Me

Point	Linear Regression	Logistic Regression
Kaam	Regression (number predict karna)	Classification (category predict karna)
Output	Koi bhi real number ($-\infty$ se $+\infty$)	Probability: 0 se 1 ke beech
Example	Weight $\rightarrow$ Height	Study hours $\rightarrow$ Pass/Fail
Hypothesis	$h_{\theta}(x) = \theta_0 + \theta_1 x$	$h_{\theta}(x) = \sigma(\theta_0 + \theta_1 x)$
Cost Function	Mean Squared Error (MSE)	Log Loss (Binary Cross-Entropy)
Minimize kaise	Gradient Descent	Gradient Descent

2. Linear Regression Ko Classification Ke Liye Kyun Nahi Use Kar Sakte?

Sabse pehla khayal yeh aa sakta hai: 'Jab Linear Regression pehle se aati hai, toh usi se Pass/Fail bhi nikal lete hain!' Tarika yeh hoga ki data par ek best fit line lagao, aur ek threshold 0.5 rakho:

Line ka output $\leq 0.5 \rightarrow 0$ (Fail) Line ka output $> 0.5 \rightarrow 1$ (Pass)

Yeh shuru me kaam karta hua dikhta hai, lekin isme **do badi problems** hain, jinki wajah se Linear Regression classification ke liye use nahi hoti:

Problem 1 — Outlier Aane Par Best Fit Line Badal Jaati Hai

Teacher ke dataset me **12 hours** padhne wala student ek outlier jaisa hai (baaki sab 2 se 7 hours ke beech hain). Linear Regression ki line har point ke error ko minimize karti hai, isliye is ek extreme point ki taraf khinch jaati hai aur **tirchhi (flat) ho jaati hai** — notes me isi ko 'New best fit line' likha gaya hai.

- Outlier ke bina line ka 0.5 wala point **4.5 hours** par hai, aur 5 hours wale student ki predicted value ≈ 0.63 aati hai (Pass).
- 12 hours wala point jodne ke baad threshold khisak kar ≈ 4.96 hours ho jaata hai, aur 5 hours wale student ki value ghat kar ≈ 0.50 reh jaati hai — yaani wo bilkul boundary par aa gaya.
- Matlab ek extreme point ne poore model ka decision badal diya. Classification me hume aisa unstable behaviour nahi chahiye.

Problem 2 — Output 1 Se Bada Ya 0 Se Chhota Ho Sakta Hai

Probability hamesha **0 se 1** ke beech hoti hai. Lekin seedhi line kahin bhi jaa sakti hai: bahut zyada study hours par output > 1 (jaise 1.5) aur bahut kam hours par output < 0 (jaise -0.4). '150% chance of passing' ya 'minus 40% chance' ka koi matlab nahi banta. Teacher ne isi ko '*squash the line*' kaha hai — humein line ko 0 aur 1 ke andar **squash** karna hai.

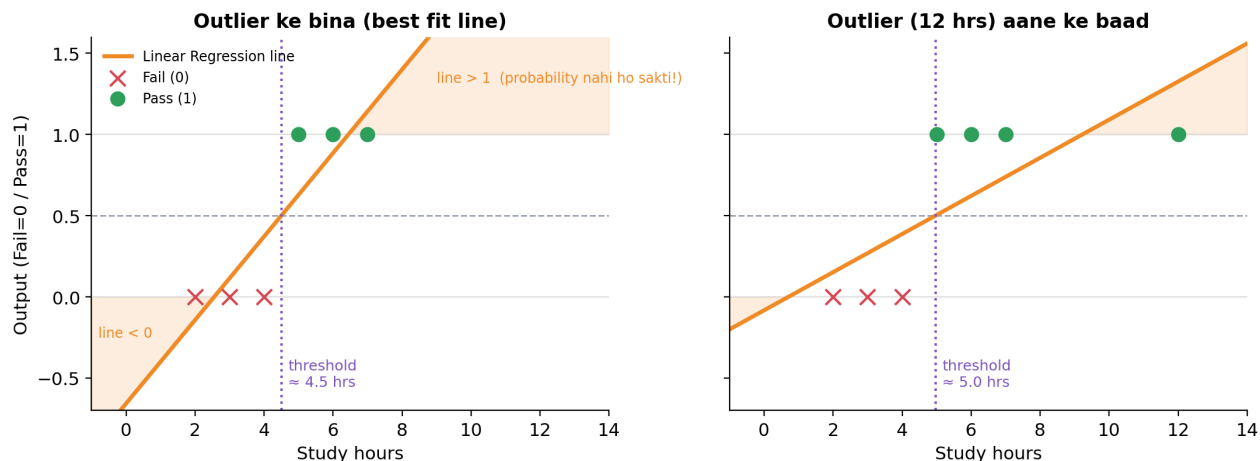


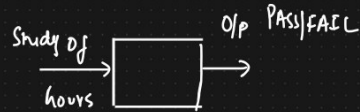
Figure 1: Left — outlier ke bina line. Right — 12 hours wala point aane par line flat ho gayi aur threshold khisak gaya. Shaded hisse 1 se upar aur 0 se neeche wale outputs dikhate hain.

Yaad rakhein: Dono problems ka ek hi solution hai — line ke output ko kisi aise function se guzaaro jo har number ko 0 aur 1 ke beech squash kar de. Wahi function hai **Sigmoid** (next section).

Logistic Regression (Binary classification) ←

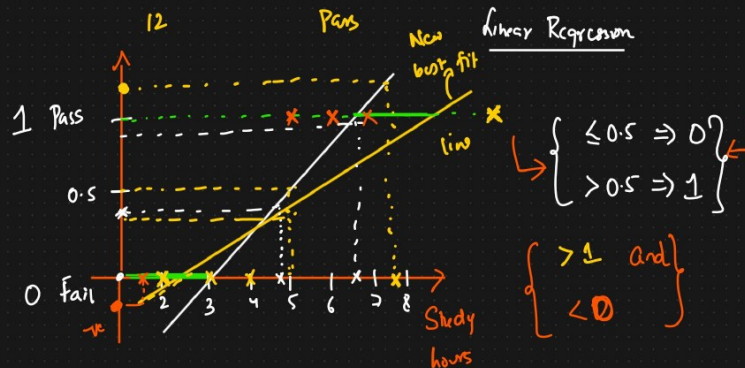
<u>Datent</u>	Pass/Fail
Study	O/p { <u>Binary categories</u> }

2	Fail
3	Fail
4	Fail
5	Pass
6	Pass
7	Pass
12	Pass



0 to 1

Logistic Regression



Why we cannot use Linear Regression for Classification?

- ① Outlier {best fit line change}
② > 1 and < 0 {Squash line}

Isme dikhaya gaya hai: Study hours ka dataset (Pass/Fail), Linear Regression ki best fit line aur outlier aane par 'New best fit line', 0.5 threshold rule, aur 'Why we cannot use Linear Regression for Classification?' ke do reasons — (1) Outlier, (2) > 1 and < 0 .

3. Logistic Regression Kaise Solve Karta Hai — Sigmoid Function

Logistic Regression ka idea bahut simple hai: pehle wahi **best fit line** nikalo, aur fir uske output ko **Sigmoid Activation Function** se guzaar do, jo kisi bhi number ko 0 se 1 ke beech me convert kar deta hai.

- **Step 1:** Linear part nikalo: $z = \theta_0 + \theta_1 x$ (bilkul Linear Regression wali line).
- **Step 2:** z ko sigmoid me daalo: $\sigma(z) = 1 / (1 + e^{-z})$.
- **Step 3:** Jo output aaya wahi hamari **hypothesis** $h_\theta(x)$ hai — yeh 0 se 1 ke beech ki probability hai.

$$h_\theta(x) = \sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{where } z = \theta_0 + \theta_1 x$$

Yahi formula teacher ke notes me 'Logistic Regression Hypothesis' ke naam se box me likha gaya hai.

Sigmoid Ki Khaas Baatein

- **Output hamesha (0, 1) ke andar** rehta hai — kabhi 0 se neeche ya 1 se upar nahi jaata. Isse Problem 2 solve ho jaati hai.
- $z = 0$ par $\sigma(z) = 0.5$ hota hai (curve ka beech ka point).
- z bahut bada positive ho toh $\sigma(z) \approx 1$; z bahut bada negative ho toh $\sigma(z) \approx 0$.
- Curve **S-shape** ka hota hai aur dono ends par flat ho jaata hai — isliye 12 hours jaisa outlier model ko zyada khinch nahi pata (Problem 1 solve).
- Function smooth hai (differentiable), isliye Gradient Descent use kar sakte hain.

z	-6	-3	-1	0	1	3	6
$\sigma(z)$	0.0025	0.047	0.269	0.5	0.731	0.953	0.9975

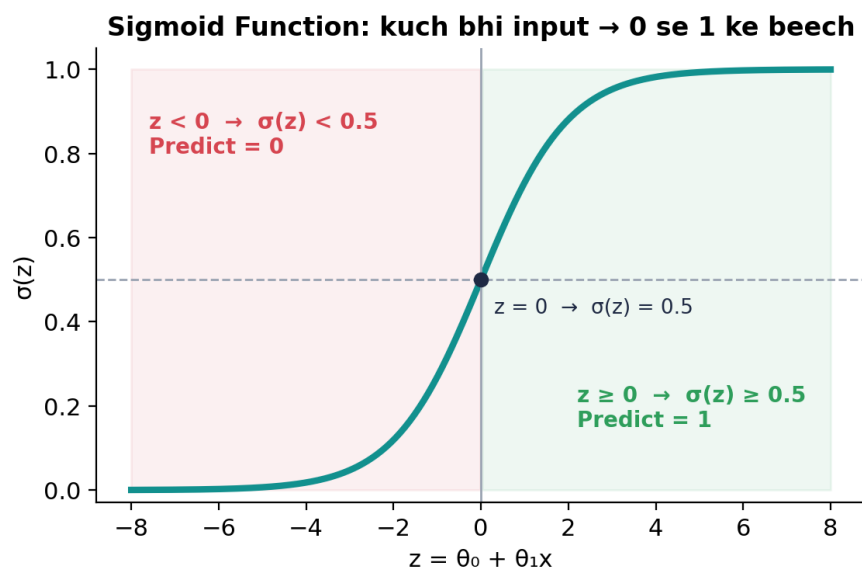


Figure 2: Sigmoid curve. $z \geq 0 \rightarrow \sigma(z) \geq 0.5 \rightarrow \text{Predict 1}$; $z < 0 \rightarrow \sigma(z) < 0.5 \rightarrow \text{Predict 0}$.

Output Ka Matlab — Probability

$h_{\theta}(x)$ ko hum $P(y = 1 | x)$ ki tarah padhte hain, yaani 'is input x ke liye output 1 hone ki probability'. Jaise $h_{\theta}(x) = 0.8$ ka matlab hai: is student ke **80% chance** hain pass hone ke. Fir hum decision rule lagate hain:

$$h_{\theta}(x) \geq 0.5 \rightarrow \text{Predict } y = 1 \text{ (Pass)} \quad h_{\theta}(x) < 0.5 \rightarrow \text{Predict } y = 0 \text{ (Fail)}$$

Kyunki $\sigma(z) \geq 0.5$ tabhi hota hai jab $z \geq 0$, isliye decision boundary wahan hai jahan $\theta_0 + \theta_1 x = 0$, yaani $x = -\theta_0 / \theta_1$.

Worked Example

Maan lo training ke baad model ne $\theta_0 = -9$ aur $\theta_1 = 2$ seekha. Toh $z = -9 + 2x$. Decision boundary $x = 9/2 = 4.5$ hours par hogi:

Study hours (x)	$z = -9 + 2x$	$\sigma(z) = P(\text{Pass})$	Prediction
3	-3	0.047	Fail (0)
4.5	0	0.500	Boundary
6	3	0.953	Pass (1)
10	11	≈ 1.000	Pass (1)

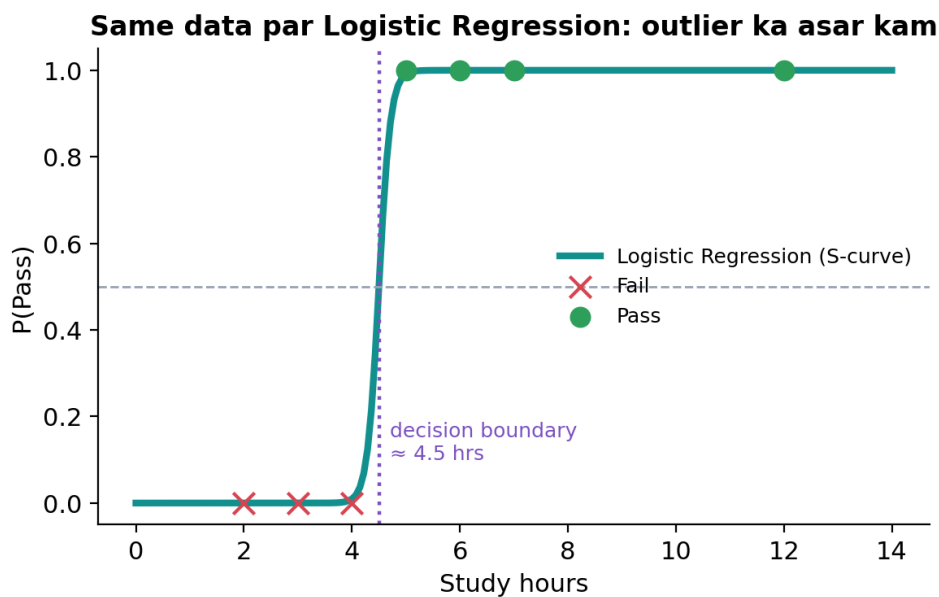


Figure 3: Wahi dataset (12 hours wale outlier ke saath) par Logistic Regression ka S-curve. Decision boundary ≈ 4.5 hours par hi rahi — outlier ne zyada asar nahi kiya.

4. Cost Function — Linear Regression Wala MSE Yahan Kyun Nahi Chalega?

Linear Regression me humne yeh cost function use kiya tha (Mean Squared Error):

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Wahan $h_{\theta}(x) = \theta_0 + \theta_1 x$ ek seedhi line thi, isliye $J(\theta)$ ka graph ek **saaf bowl-shape (Convex function)** banta tha jisme sirf ek hi minimum hota hai — **Global Minimum**. Gradient Descent kahin se bhi shuru ho, wahi pahunch jaata tha.

Ab agar Logistic Regression me bhi wahi MSE lagayein, lekin $h_{\theta}(x) = 1 / (1 + e^{-z})$ (sigmoid) ke saath, toh problem aati hai: sigmoid ek **non-linear** function hai, aur jab usko square ke andar daalte hain toh $J(\theta)$ ka graph **wavy / non-convex** ban jaata hai — jisme **kai Local Minima** aur flat patches aa jaate hain (teacher ke notes ka right-side graph).

	Linear Regression + MSE	Logistic Regression + MSE (sigmoid ke saath)
$h_{\theta}(x)$	Seedhi line (linear)	Sigmoid (non-linear)
$J(\theta)$ ka shape	Convex — ek bowl	Non-convex — wavy, kai gaddhe
Minimum	Sirf ek Global Minimum	Local Minima + Global Minimum
Gradient Descent	Hamesha global minimum tak pahunchta hai	Local minimum me atak sakta hai

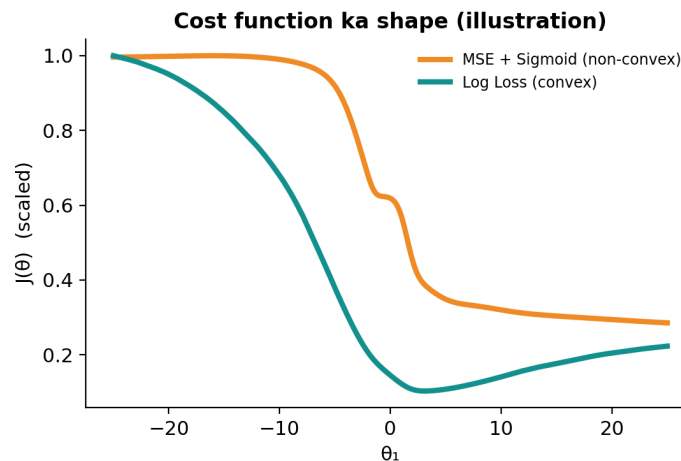
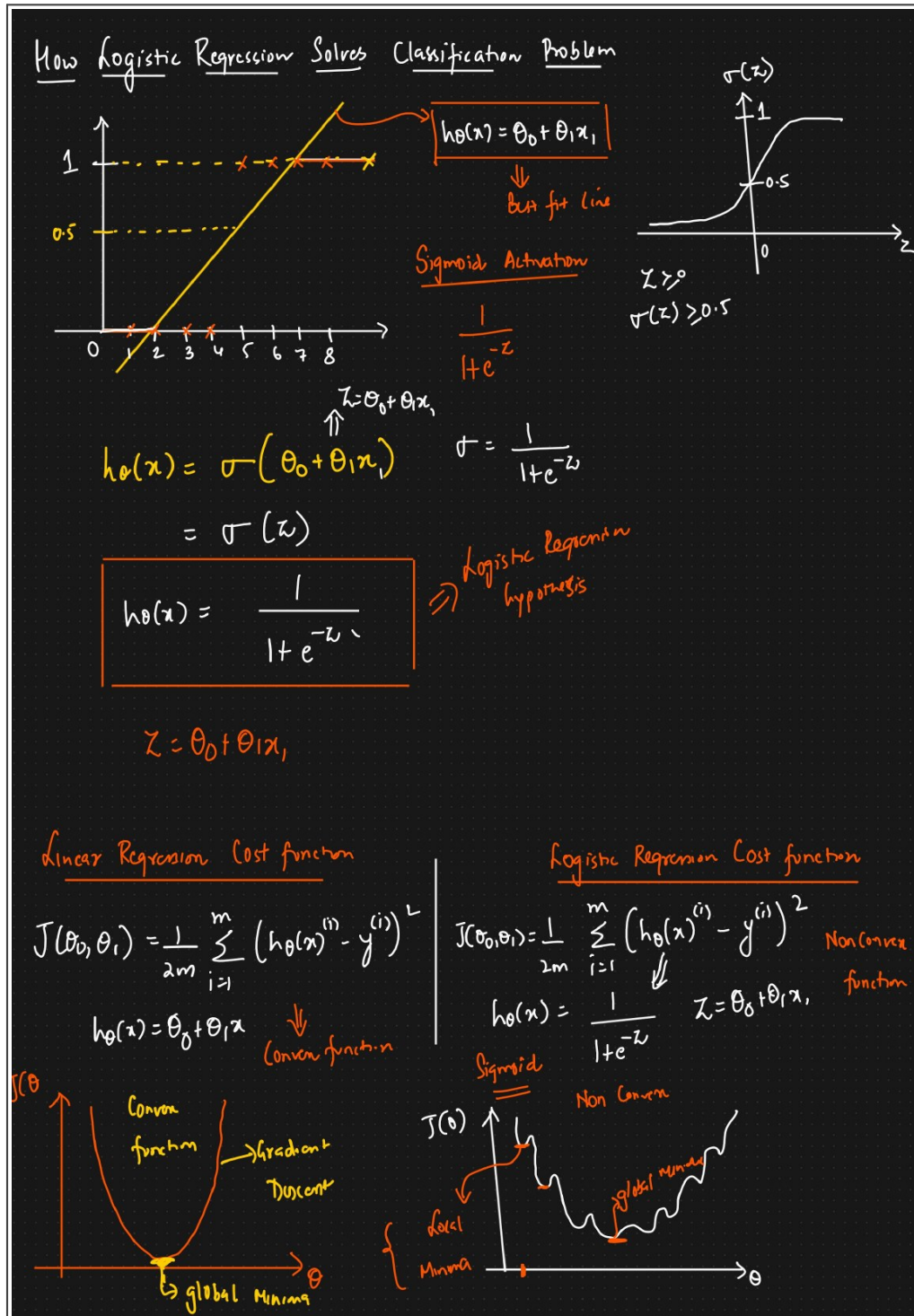


Figure 4: MSE + Sigmoid ka $J(\theta)$ — flat patches aur kink (non-convex). Comparison ke liye Log Loss ka smooth curve bhi dikhaya gaya hai.

Yaad rakhein: Isliye Logistic Regression ke liye ek naya cost function chahiye jo sigmoid ke saath bhi **convex** rahe. Woh hai **Log Loss**.



Isme dikhaya gaya hai: Linear Regression ki line ko Sigmoid Activation se squash karna, Logistic Regression Hypothesis $h_0(x) = 1/(1+e^{-z})$, sigmoid graph ($z \geq 0 \rightarrow \sigma(z) \geq 0.5$), aur Linear vs Logistic cost function ka comparison — convex bowl vs non-convex function (local minima).

5. Log Loss (Binary Cross-Entropy) — Logistic Regression Ka Cost Function

MSE wale J me jo $(h_{\theta}(x) - y)^2$ wala part hai, usko hum ek alag naam dete hain — **Cost** $(h_{\theta}(x), y)$, jo **ek single example** ka loss batata hai. Phir $J(\theta)$ us loss ka average hota hai. Logistic Regression me is Cost ki definition **Log Loss** se badal dete hain:

$$\text{Cost}(h_{\theta}(x), y) = -\log(h_{\theta}(x)) \text{ if } y = 1 \mid -\log(1 - h_{\theta}(x)) \text{ if } y = 0$$

Intuition — Yeh Formula Kaam Kaise Karta Hai?

- **Jab actual y = 1:** loss = $-\log(h)$. Agar model ne $h \approx 1$ (sahi aur confident) bola toh $-\log(1) = 0 \rightarrow$ koi penalty nahi. Agar model ne $h \approx 0$ (galat) bola toh $-\log(0) \rightarrow \infty \rightarrow$ **bahut badi penalty**.
- **Jab actual y = 0:** loss = $-\log(1 - h)$. Ab $h \approx 0$ sahi hai (loss ≈ 0) aur $h \approx 1$ galat hai (loss $\approx \infty$).
- Yaani model jitna zyada **confident hokar galat** hoga, utni hi badi saza milegi. Yahi behaviour hume chahiye.

Actual y	Model ka $h_{\theta}(x)$	Formula	Loss	Matlab
1	0.90	$-\log(0.90)$	0.105	Sahi aur confident $\rightarrow$ chhota loss
1	0.50	$-\log(0.50)$	0.693	Confused $\rightarrow$ medium loss
1	0.10	$-\log(0.10)$	2.303	Galat $\rightarrow$ bada loss
1	0.01	$-\log(0.01)$	4.605	Bahut galat $\rightarrow$ bahut bada loss
0	0.10	$-\log(1-0.10)$	0.105	Sahi aur confident $\rightarrow$ chhota loss
0	0.90	$-\log(1-0.90)$	2.303	Galat $\rightarrow$ bada loss

(Yahan log ka matlab natural logarithm ln hai, jo ML me standard hai.)

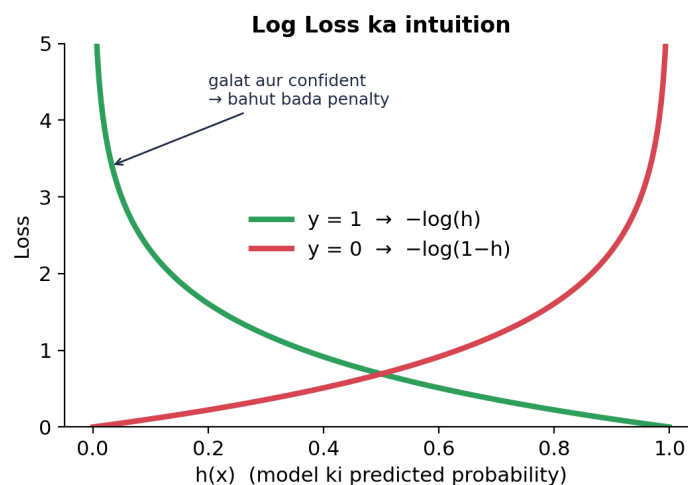


Figure 5: $y = 1$ ke liye $-\log(h)$ aur $y = 0$ ke liye $-\log(1-h)$. Galat aur confident prediction par loss tezi se badhta hai.

Do Cases Ko Ek Formula Me Jodna

Do alag-alag cases likhna asaan nahi hota, isliye inhe ek hi formula me likh dete hain. Trick yeh hai ki y sirf 0 ya 1 hota hai:

$$\text{Cost}(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$$

- $y = 1$ ho toh $(1 - y) = 0$, dusra term gayab $\rightarrow \text{Cost} = -\log(h)$.
- $y = 0$ ho toh pehla term gayab $\rightarrow \text{Cost} = -\log(1 - h)$.

Ab saare m training examples ka average lene par **final Logistic Regression cost function** milta hai:

$$J(\theta_0, \theta_1) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

Dhyaan dein: Teacher ke page par final formula me $1/2m$ likha hai aur bracket ke andar dusre term se pehle **minus (-)** hai. Standard form me $1/m$ hota hai aur bracket ke andar dono terms **plus (+)** se jude hote hain (bahar wala minus poore bracket par lagta hai). $1/2m$ aur $1/m$ me sirf ek constant ka farak hai jo minimum ki position nahi badalta, par sign wala part sahi likhna zaroori hai — exam me upar wala standard form hi likhein.

Worked Example

Teen training examples lo:

Example	Actual y	Predicted $h_{\theta}(x)$	Loss formula	Loss
1	1	0.90	$-\log(0.90)$	0.105
2	0	0.20	$-\log(1 - 0.20)$	0.223
3	1	0.60	$-\log(0.60)$	0.511

$J(\theta) = (0.105 + 0.223 + 0.511) / 3 = 0.839 / 3 \approx \mathbf{0.28}$. Training ka kaam hai isi J ko aur chhota banana.

Log Loss Convex Kyun Hai — Isse Kya Fayda?

Log Loss ko sigmoid ke saath combine karne par $J(\theta)$ ka shape wapas ek **convex bowl** ban jaata hai (Figure 4 me Log Loss wala smooth curve). Matlab **sirf ek Global Minimum** aur Gradient Descent kahin atkega nahi. Teacher ne yahi 'Convex function' likh kar batayi hai.

6. Gradient Descent — Convergence Algorithm

Goal wahi hai jo Linear Regression me tha: **Cost function $J(\theta_0, \theta_1)$ ko minimize karna, θ_0 aur θ_1 ko change karke.** Iske liye wahi **Convergence Algorithm (Gradient Descent)** use hota hai:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \quad (j=0 \text{ and } 1)$$

- **$:=$** matlab purani value ko naye value se replace karo.
- **α (alpha) — Learning Rate:** ek chhota positive number (jaise 0.001 ya 0.1) jo har step ka size decide karta hai.
- **$\partial J / \partial \theta_j$** — J ka slope (gradient). Slope negative ho toh θ badhta hai, positive ho toh θ ghatta hai — dono cases me θ minimum ki taraf hi jaata hai.
- θ_0 aur θ_1 dono ko **simultaneously** update karte hain (pehle dono naye values calculate karo, fir ek saath assign karo).

Derivative Nikalne Par Final Update Rules

Log Loss ke saath sigmoid ki derivative nikalne par bahut hi clean result milta hai — aur dekhne me wahi hai jo Linear Regression me tha:

$$\theta_0 := \theta_0 - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Farak bas itna hai ki yahan **$h_{\theta}(\mathbf{x}) = 1 / (1 + e^{-(\theta_0 + \theta_1 \mathbf{x})})$** sigmoid hai, jabki Linear Regression me $h_{\theta}(x) = \theta_0 + \theta_1 x$ tha. θ_1 wale update me extra $\mathbf{x}^{(i)}$ isliye multiply hota hai kyunki θ_1 hamesha x ke saath multiply hota hai.

Learning Rate α Ka Asar

- **α bahut chhota:** convergence bahut slow — bahut zyada iterations lagenge.
- **α sahi:** J aaram se kam hota hua global minimum par settle ho jaata hai.
- **α bahut bada:** algorithm minimum ko overshoot kar deta hai, idhar-udhar uchhalta hai ya diverge bhi kar sakta hai.

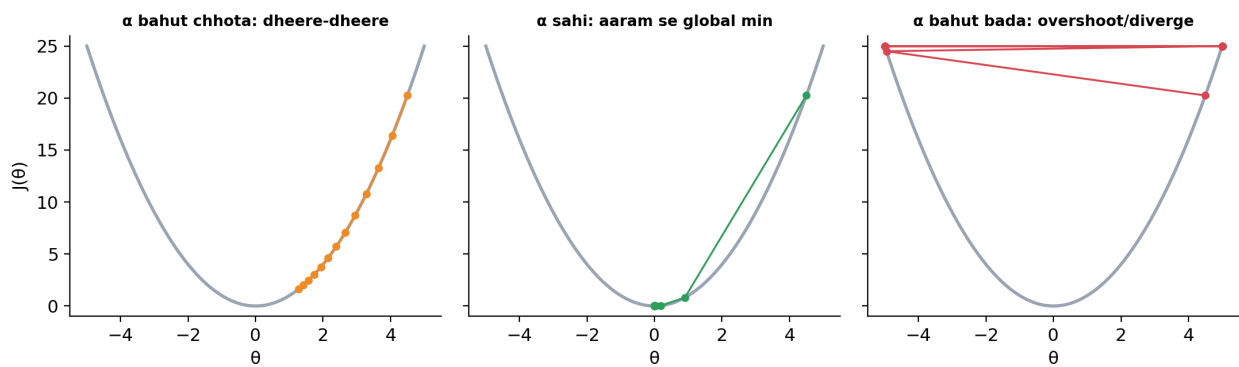


Figure 6: Gradient Descent par learning rate ka asar (simple bowl $J(\theta) = \theta^2$ par illustration).

Ek Iteration Ka Worked Example

Study-hours dataset lo ($m = 7$) aur shuruaat me $\theta_0 = 0, \theta_1 = 0, \alpha = 0.1$ rakho. Tab $z = 0 \rightarrow h_\theta(x) = \sigma(0) = 0.5$ har student ke liye. Error = $h - y$:

x (hours)	2	3	4	5	6	7	12	Sum
y	0	0	0	1	1	1	1	
$h - y$	0.5	0.5	0.5	-0.5	-0.5	-0.5	-0.5	-0.5
$(h - y) \cdot x$	1.0	1.5	2.0	-2.5	-3.0	-3.5	-6.0	-10.5

- $\theta_0 := 0 - 0.1 \times (1/7) \times (-0.5) = \mathbf{0.0071}$
- $\theta_1 := 0 - 0.1 \times (1/7) \times (-10.5) = \mathbf{0.15}$

Ek hi iteration ke baad J ki value $\ln 2 \approx 0.693$ se ghat kar $\approx \mathbf{0.575}$ ho jaati hai. Yeh process baar-baar repeat hota hai jab tak J aur kam hona band na ho jaaye (convergence). Final θ_0, θ_1 hamare trained Logistic Regression model ko define karte hain.

Poore Logistic Regression Ka Summary

- Step 1:** Dataset lo (X = features, $y = 0/1$ labels).
- Step 2:** $z = \theta_0 + \theta_1 x$ nikalo aur sigmoid lagao: $h_\theta(x) = \sigma(z)$.
- Step 3:** Log Loss wala cost function $J(\theta)$ define karo.
- Step 4:** Gradient Descent se θ_0, θ_1 ko baar-baar update karo jab tak J converge na ho.
- Step 5:** Naye input ke liye $h_\theta(x)$ nikalo; ≥ 0.5 ho toh class 1, warna class 0.

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \underbrace{(h_{\theta}(x^{(i)}) - y^{(i)})^2}_{\text{Cost}(h_{\theta}(x^{(i)}), y^{(i)})}$$

$h_{\theta}(x^{(i)}) = \frac{1}{1+e^{-z}}, z = \theta_0 + \theta_1 x_i$

↓
this denote

{ log loss }

$$\text{Cost}(h_{\theta}(x^{(i)}), y^{(i)}) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y=1 \\ -\log(1-h_{\theta}(x)) & \text{if } y=0 \end{cases}$$

↓
convex function

$$\text{Cost}(h_{\theta}(x^{(i)}), y^{(i)}) = -y \log(h_{\theta}(x)) - (1-y) \log(1-h_{\theta}(x))$$

$$J(\theta_0, \theta_1) = -\frac{1}{2m} \sum_{i=1}^m (y^{(i)} \log(h_{\theta}(x^{(i)})) - (1-y^{(i)}) \log(1-h_{\theta}(x^{(i)})))$$

Minimize Cost function $J(\theta_0, \theta_1)$ by changing θ_0 & θ_1

Convergence Algorithm

Repeat

{

$J < 0 \text{ and } 1$
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$

}

Isme dikhaya gaya hai: MSE ke $(h - y)^2$ ko $\text{Cost}(h_{\theta}(x), y)$ naam dena, Log Loss ke do cases ($-\log(h)$ if $y=1$, $-\log(1-h)$ if $y=0$), unko ek formula me jodna, final $J(\theta_0, \theta_1)$, aur Gradient Descent (Convergence Algorithm).

7. Logistic Regression for Multi-Class — One-vs-Rest (OvR)

Ab tak humne sirf **do classes** (Pass/Fail) wale problems dekhe. Lekin real life me aksar **teen ya zyada classes** hoti hain — jaise handwritten digit (0 se 9, yaani 10 classes), iris flower ki 3 species, ya fruit type (Apple / Banana / Orange). Logistic Regression by design sirf **binary** hai (sigmoid ek hi probability deta hai), toh multi-class problem ko kaise solve karein?

Iska ek simple aur popular tarika hai **One-vs-Rest (OvR)**, jise **One-vs-All (OvA)** bhi kehte hain.

7.1 Core Idea

Ek multi-class problem ko K alag-alag binary problems me tod do. Agar K classes hain toh K alag Logistic Regression classifiers train karo. **Classifier k** ka sawaal sirf yeh hota hai: 'Kya yeh sample class k ka hai, ya baaki sab (Rest) me se kisi ka?'

- 3 classes (A, B, C) hain → **3 classifiers: A vs Rest, B vs Rest, C vs Rest.**
- 10 classes (digits 0–9) hain → **10 classifiers.**

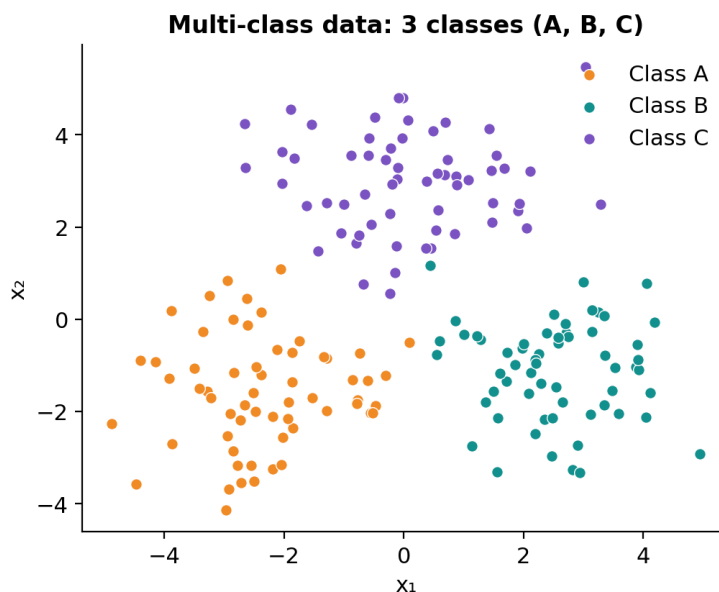


Figure 7: Ek 3-class dataset (A, B, C) jisme har class ke do features x_1 aur x_2 hain.

7.2 Training — Step by Step

- **Step 1:** Original data me 3 classes A, B, C hain.
- **Step 2 (Relabel):** Har classifier ke liye labels badal do — jo class target hai use **1** aur baaki sab ko **0** bana do.
- **Step 3:** Har relabeled dataset par ek normal **binary Logistic Regression** train karo (sigmoid + log loss + gradient descent, bilkul upar jaisa).
- **Step 4:** Is tarah hume K trained models mil jaate hain, har model ka apna θ (weights) hota hai.

Chhota example — 5 samples ka relabeling:

Sample	Original label	Classifier 1 (A vs Rest)	Classifier 2 (B vs Rest)	Classifier 3 (C vs Rest)
s1	A	1	0	0
s2	B	0	1	0
s3	C	0	0	1
s4	A	1	0	0
s5	C	0	0	1

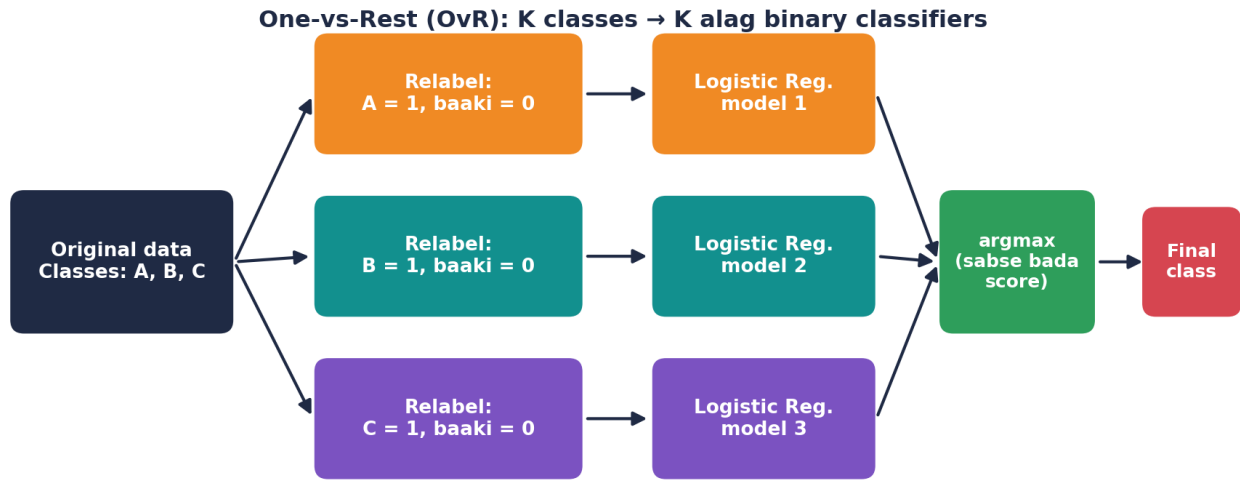


Figure 8: OvR ka poora flow — data relabel hota hai, K classifiers train hote hain, aur prediction ke waqt argmax final class chunta hai.

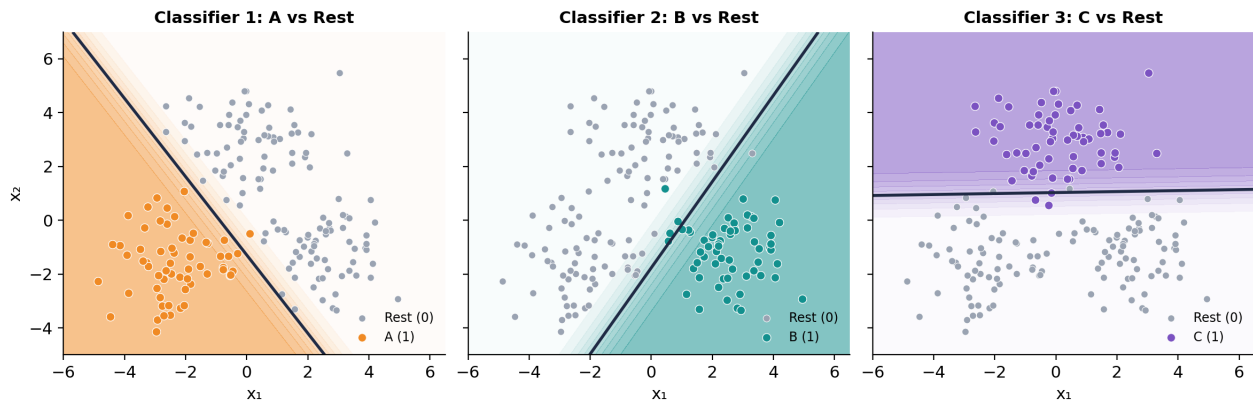


Figure 9: Teeno binary classifiers. Har plot me target class rangeen hai aur baaki sab grey (Rest). Dark line $P = 0.5$ wali decision boundary hai, aur shading us classifier ki probability dikhati hai.

7.3 Prediction — argmax Ka Kaam

Naya data point aane par:

- **Step 1:** Teeno classifiers me x daalo. Har classifier ek probability deta hai: $h^{(A)}(x)$, $h^{(B)}(x)$, $h^{(C)}(x)$ — 'x apni class ka hone ka kitna chance hai'.
- **Step 2:** Jis classifier ka score **sabse bada** ho, uski class ko final answer bana do.

$$\hat{y} = \operatorname{argmax}_{k \in \{1, \dots, K\}} h_{\theta}^{(k)}(x)$$

Yahan **argmax** ka matlab hai 'wo k dhoondo jiske liye value maximum ho'. Ise max se alag samjho: *max* value batata hai, *argmax* batata hai **kaunsi class** ne jeeta.

Worked Example

Maan lo ek naya point aaya jiske teeno classifiers ne yeh scores diye:

Classifier	Sawaal	Score (probability)
1	A vs Rest	0.00
2	B vs Rest	0.30
3	C vs Rest	0.53 ← sabse bada

Sabse bada score C ka hai (0.53), isliye **final prediction = Class C**. Dhyaan rahe ki teeno scores ka sum 1 nahi hota, kyunki har classifier *alag se* apna binary sawaal solve karta hai. Agar sum = 1 wali probabilities chahiye ho toh scores ko unke total se divide (normalize) kar sakte hain: yahan $0.30/0.83 \approx 0.36$ (B) aur $0.53/0.83 \approx 0.64$ (C). scikit-learn ka `OneVsRestClassifier.predict_proba` yeh normalization khud kar deta hai.

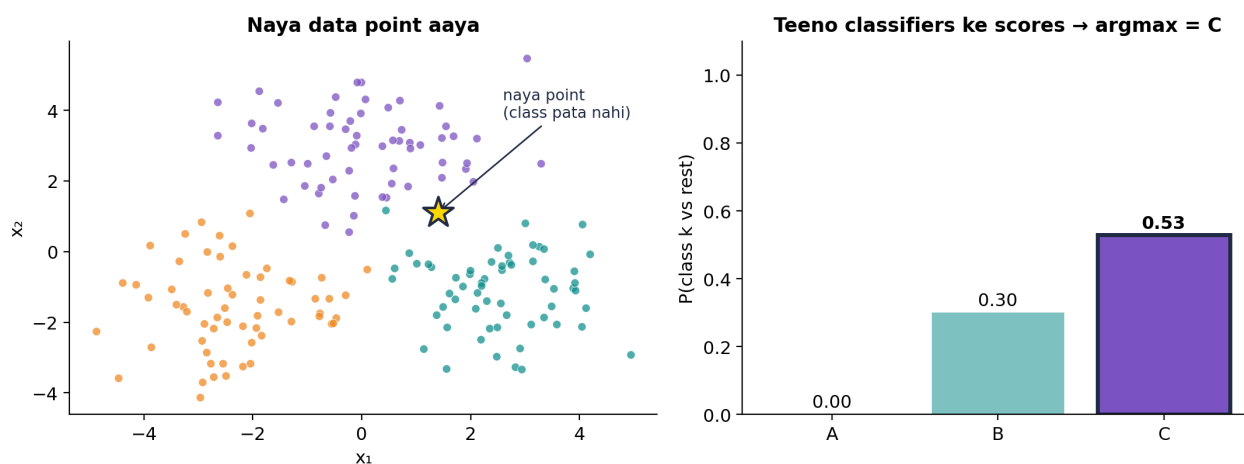


Figure 10: Left — naya point (star) teeno clusters ke beech aaya. Right — teeno classifiers ke scores; sabse bada bar (C) jeet gaya.

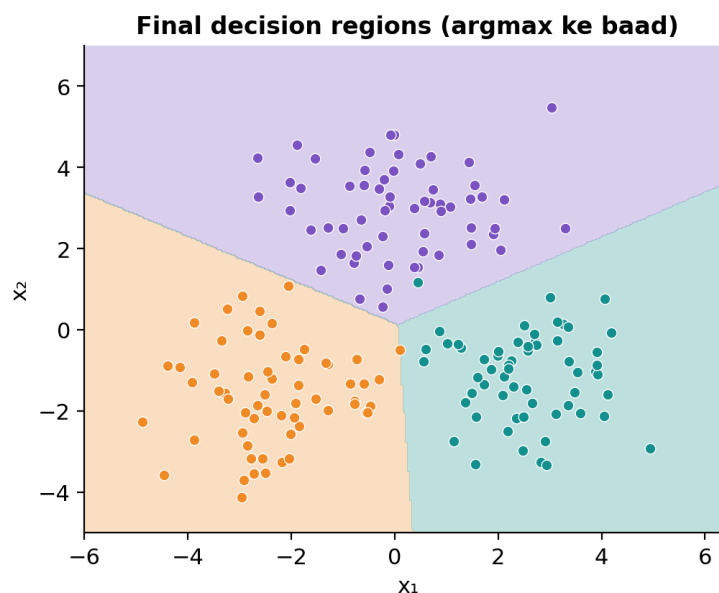


Figure 11: Teeno classifiers ko combine karne par bane final decision regions. Har region wahi hai jahan us class ka score sabse bada hai.

7.4 Ambiguity Kaise Solve Hoti Hai?

Kyunki classifiers alag-alag train hue hain, kabhi-kabhi **do classifiers ek saath 'haan'** bol sakte hain (dono ka score > 0.5), ya **koi bhi haan nahi** bolta (sabka score < 0.5). Dono situations me **argmax** hi tie-breaker ka kaam karta hai — hum 0.5 threshold ki jagah relative comparison karte hain aur sabse bada score jeet jaata hai.

7.5 Python Code (scikit-learn)

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.multiclass import OneVsRestClassifier

X, y = load_iris(return_X_y=True)          # 3 classes
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

ovr = OneVsRestClassifier(LogisticRegression(max_iter=1000))
ovr.fit(X_train, y_train)

print(len(ovr.estimators_))                # 3 -> K = 3 classifiers
print(ovr.score(X_test, y_test))           # ~0.93 accuracy
print(ovr.predict_proba(X_test[:1]))       # normalized scores (sum = 1)
print(ovr.predict(X_test[:1]))             # class with the highest score
```

Naye scikit-learn versions me LogisticRegression khud multi-class ke liye default me 'multinomial' (softmax) use karta hai, isliye OvR explicitly chahiye ho toh OneVsRestClassifier wrapper use karo.

7.6 Advantages Aur Limitations

Advantages	Limitations
Bahut simple — kisi bhi binary classifier se multi-class ban jaata hai	K classifiers train karne padte hain (10 classes = 10 models)
Har class ka apna model hone se interpret karna aasan (kaun si class ke liye kaun se features important)	Har binary problem imbalanced hoti hai: 1 class vs (K – 1) classes
OvO ke muqable kam models: K vs $K(K-1)/2$	Scores alag-alag classifiers ke hain, isliye calibrated probabilities guaranteed nahi hoti
Parallel me train kar sakte hain	Overlap wali classes me confusion ho sakta hai

7.7 OvR vs OvO vs Softmax — Quick Comparison

	One-vs-Rest (OvR)	One-vs-One (OvO)	Softmax (Multinomial)
Kitne models	K	$K(K-1)/2$	1 model (K outputs)
Har model ka data	Poora data (1 vs baaki)	Sirf 2 classes ka data	Poora data
Prediction	argmax of K scores	Voting	argmax of softmax probabilities
3 classes ke liye	3 models	3 models	1 model
10 classes ke liye	10 models	45 models	1 model

Yaad rakhein: OvR ko aise yaad rakho — '**K classes = K binary questions: kya yeh class k hai ya nahi?**' Prediction me sabse confident classifier jeetta hai.

Part 2 — Performance Metrics

Classification model kitna achha hai — yeh naapne ke tareeke: Confusion Matrix, Accuracy, Precision, Recall aur F-Beta Score.

8. Performance Metrics Kyun Zaroori Hain?

Model train karne ke baad sabse bada sawaal hota hai: '**Yeh model kitna achha kaam kar raha hai?**' Iska answer dene ke liye hume **performance metrics** chahiye hote hain. Is topic me hum yeh 5 cheezein padhenge:

- **1. Confusion Matrix**
- **2. Accuracy**
- **3. Precision**
- **4. Recall**
- **5. F-Beta Score**

Teacher ke page ke top-right me R-squared aur Adjusted R-squared bhi likhe hain — woh **Regression** ke metrics hain (Linear Regression ke liye). Classification (Logistic Regression) ke liye upar wale 5 metrics use hote hain.

Dataset — Actual Vs Predicted

Ek trained Logistic Regression model ko test dataset par chalane par har row ke liye do values milti hain: y = asli (actual) output, aur $\hat{y}$ (**y-hat**) = jo model ne predict kiya. Teacher ka example dataset (7 rows) yeh hai:

Row	Actual y	Predicted $\hat{y}$	Result
1	0	1	Galat — FP
2	1	1	Sahi — TP
3	0	0	Sahi — TN
4	1	1	Sahi — TP
5	1	1	Sahi — TP
6	0	1	Galat — FP
7	1	0	Galat — FN

Ab is table ko compact tarike se summarize karne ke liye hum **Confusion Matrix** banate hain.

9. Confusion Matrix

Confusion Matrix ek 2×2 table hai jo dikhati hai ki model ne kitni predictions **sahi** aur kitni **galat** ki, aur galat kis type ki. Iske chaar cells hote hain:

Term	Full Form	Actual	Predicted	Matlab
TP	True Positive	1	1	Positive tha, model ne positive bola — sahi
TN	True Negative	0	0	Negative tha, model ne negative bola — sahi
FP	False Positive (Type I error)	0	1	Negative tha, par model ne positive bol diya — galat (false alarm)
FN	False Negative (Type II error)	1	0	Positive tha, par model ne negative bol diya — galat (miss)

Yaad rakhein: Naam padhne ki trick: **Pehla shabd (True/False)** batata hai ki model ka prediction *sahi tha ya galat*. **Dusra shabd (Positive/Negative)** batata hai ki model ne *kya predict kiya*. Jaise **False Positive** = model ne 'Positive' predict kiya aur woh 'False' (galat) tha.

Upar ke 7-row example ko count karein: **TP = 3** (rows 2, 4, 5), **FP = 2** (rows 1, 6), **FN = 1** (row 7), **TN = 1** (row 3). Teacher ki matrix me isi tarah rows = Predicted values aur columns = Actual values rakhe gaye hain:

		ACTUAL values	
		1	0
PREDICTED values	1	TP 3	FP 2
	0	FN 1	TN 1

$$\text{Accuracy} = (3+1)/(3+2+1+1) = 4/7 \approx 57\%$$

Figure 12: Teacher wale example ka Confusion Matrix (rows = Predicted, columns = Actual). Hare cells sahi predictions hain, laal cells galat.

Dhyaan dein: Do baatein: (1) Teacher ki pehli matrix me top-left cell me '2' circle kiya gaya hai, lekin agle step me accuracy nikalte waqt **TP = 3** use hua hai, aur data count karne par bhi TP = 3 aata hai — isliye 3 hi sahi hai. (2) Teacher ki layout (rows = Predicted, columns = Actual) **scikit-learn ki layout se ulti hai**. sklearn ka `confusion_matrix(y_true, y_pred)` rows = Actual aur columns = Predicted deta hai, order [0, 1]: **[[TN, FP], [FN, TP]]**. Isi example par sklearn **[[1, 2], [1, 3]]** dega. Exam me jo layout poochi jaaye, uska labelling dhyaan se karein.

10. Accuracy

Accuracy sabse simple metric hai: **kul predictions me se kitni sahi thi**. Sahi predictions wo hain jo TP ya TN me aati hain:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

Teacher ke example me: $\text{Accuracy} = (3 + 1) / (3 + 2 + 1 + 1) = 4/7 \approx 57.1\%$.

Accuracy Kab Dhokha Deti Hai? — Imbalanced Dataset

Maan lo ek binary classification dataset me **1000 datapoints** hain: **900** class 1 ke aur sirf **100** class 0 ke. Ise **Imbalanced Dataset** kehte hain (ek class bahut zyada, ek bahut kam).

Ab ek bewakoof model socho jo input dekhe bina **hamesha '1' bol deta hai**. Iski accuracy $900/1000 = 90\%$ ho jaayegi — sunne me bahut achha! Lekin sach yeh hai ki model ne class 0 ka ek bhi example pakda hi nahi:

		ACTUAL values	
		1	0
PREDICTED values	1	TP 900	FP 100
	0	FN 0	TN 0

Accuracy = $900/1000 = 90\%$, par class 0 ek bhi pakda nahi!

Figure 13: Imbalanced dataset — 'hamesha 1' predict karne wala model 90% accuracy dikhata hai, par $TN = 0$ (class 0 ek bhi sahi nahi).

- Real life me fraud detection, disease detection, spam filtering jaise problems aksar imbalanced hote hain (fraud 1% hota hai, 99% normal).
- Aise me **sirf accuracy par bharosa nahi karna chahiye** — isiliye humein **Precision, Recall aur F-Beta** chahiye.

Performance Metrics, Accuracy, Precision, Recall And F-Beta

Topics to be covered

- ① Confusion matrix
- ② Accuracy
- ③ Precision
- ④ Recall
- ⑤ F-Beta Score

① Confusion Matrix

Actual Values

	1	0
1	3	2
0	1	1

←

Predicted Values

	1	0
1	TP	FP
0	FN	TN

Accuracy = $\frac{TP+TN}{TP+FP+FN+TN}$

= $\frac{3+1}{3+2+1+1}$

= $\frac{4}{7}$

② Dataset

Binary classification

1000 datapoints

900 → 1

100 → 0

↓

90% Accuracy

Imbalanced Dataset

Logistic Regression

R squared

Adjusted R squared

Isme dikhaya gaya hai: Topics to be covered (Confusion Matrix, Accuracy, Precision, Recall, F-Beta), dataset me actual y aur predicted $\hat{y}$, Confusion Matrix (TP, FP, FN, TN), Accuracy = $(TP+TN)/(TP+FP+FN+TN) = 4/7$, aur 1000 datapoints (900 : 100) wala Imbalanced Dataset example.

11. Precision Aur Recall

Dono metrics **positive class** par focus karte hain, lekin alag-alag sawaal poochte hain:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

	Precision	Recall
Formula	$TP / (TP + FP)$	$TP / (TP + FN)$
Sawaal	Model ne jitne ko positive bola , unme se kitne sach me positive the?	Jitne sach me positive the , unme se kitne model ne pakde?
Denominator kya hai	Predicted Positive (TP + FP)	Actual Positive (TP + FN)
Kaun sa error kam karta hai	FP (false alarm)	FN (miss)
Mnemonic	'Jo bola, wo kitna sahi'	'Jo tha, wo kitna pakda'

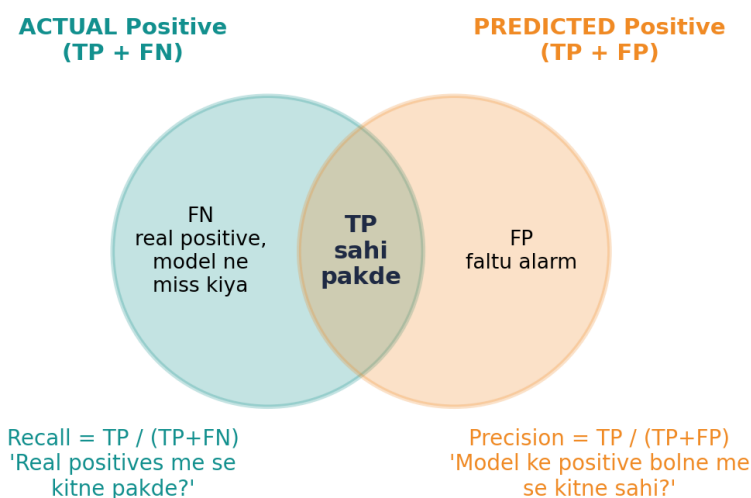


Figure 14: Precision aur Recall ek hi TP (overlap) ko alag-alag denominator se divide karte hain.

Dhyaan dein: Teacher ke notes me Precision ke saath 'Out of all the *actual* value how many are correctly predicted' aur Recall ke saath 'Out of all the *predicted* value how many are correctly predicted' likha hai. Yeh dono descriptions **aapas me swap ho gaye hain**. Formulas sahi hain (Precision = $TP / (TP + FP)$, Recall = $TP / (TP + FN)$) aur upar ki table sahi wording hai: **Precision** → **predicted positives me se sahi kitne**; **Recall** → **actual positives me se pakde kitne**. Exam me yahi wording likhna.

Example — Teacher Wale Dataset Par

- **Precision** = $TP / (TP + FP) = 3 / (3 + 2) = 3/5 = 0.60$ — model ne 5 baar 'positive' bola, unme se 3 sahi the.
- **Recall** = $TP / (TP + FN) = 3 / (3 + 1) = 3/4 = 0.75$ — asli 4 positives me se model ne 3 pakde.

Yaad rakhein: Precision aur Recall me aksar **trade-off** hota hai: model ko zyada 'strict' banao (threshold badhao) toh Precision badhti hai par Recall ghatti hai; 'lenient' banao (threshold ghatao) toh Recall badhti hai par Precision ghatti hai.

12. Use Case 1 — Spam Classification (Precision)

Spam detection me positive class = **Spam (1)** aur negative class = **Not Spam (0)**. Chaar possibilities dekhte hain:

Actual Mail	Model ne kya bola	Type	Result
Spam	Spam	TP	Good — spam sahi pakda gaya
Not Spam	Not Spam	TN	Good
Spam	Not Spam	FN	Chhota nuksan — ek spam mail inbox me aa gayi, user khud delete kar dega
Not Spam	Spam	FP	Blunder! — zaroori mail (interview call, OTP, offer letter) spam folder me chali gayi

Yahan sabse bada nuksan **False Positive (FP)** hai, isliye hume **FP ko kam karna** hai. FP kam karne wala metric hai **Precision** = $TP / (TP + FP)$. Jitna FP kam, utni Precision zyada. Isliye spam classification me **Precision** par focus karte hain.

13. Use Case 2 — Diabetes Prediction (Recall)

Ab ek person ko test karke predict karna hai ki use **diabetes hai ya nahi**. Positive class = **Diabetes (1)**.

Truth (asli)	Model ne kya bola	Type	Result
Diabetes hai	Diabetes hai	TP	Good — patient ka ilaaj shuru ho jaayega
Diabetes hai	Diabetes nahi hai	FN	Blunder! — bimar insaan ko 'healthy' bol diya, ilaaj nahi hoga, bimari badh sakti hai
Diabetes nahi	Diabetes hai	FP	Chhota nuksan — 2nd opinion / extra test se check ho jaayega
Diabetes nahi	Diabetes nahi	TN	Good

Yahan sabse bada nuksan **False Negative (FN)** hai, isliye hume **FN ko kam karna** hai. FN kam karne wala metric hai **Recall** = $TP / (TP + FN)$. Isliye disease detection jaise use cases me **Recall** sabse important hota hai.

① Precision = $\frac{TP}{TP+FP}$ } Out of all the actual value
how many are correctly predicted

	1	0	Actual
1	TP	FP	
0	FN	TN	
	Predicted		

② Recall = $\frac{TP}{TP+FN}$ } Out of all the predicted value
how many are correctly predicted

Use case ①

Spam classification

	Spam	Not Spam	Actual
Spam	TP	FP ✓	
Not Spam	FN	TN	

↓

Precision = $\frac{TP}{TP+FP}$

Mail → Spam } Good
Model → Spam }
Mail → Not Spam } Blunder
Model → Spam } Blunder

Use case 2

To predict whether person has diabetes or not

✓ Truth → diabetes }
✓ Model → Doesn't diabetes } Blunder

Truth → diabetes } good
Model → " } good

	Diabetes	No Diabetes
Diabetes	TP	FP
No Diabetes	FN	TN

↓
use case of diseases

Recall = $\frac{TP}{TP+FN}$

Isme dikhaya gaya hai: Precision aur Recall ke formulas, Use case 1 — Spam classification (FP blunder hai, isliye Precision), aur Use case 2 — Diabetes prediction (FN blunder hai, isliye Recall). Diabetes ka aakhri case (Truth: Not diabetes, Model: Diabetes) agle page par hai.

14. Assignment — Kal Stock Market Crash Hoga Ya Nahi?

Teacher ne ek assignment diya hai: 'Tomorrow the stock market will crash or not' — is problem me humein **FP kam** karna chahiye ya **FN kam**? Ise same tareeke se solve karte hain. Positive class = **Market crash hoga (1)**.

Actual	Model ne kya bola	Type	Kya hoga?
Crash hoga	Crash hoga	TP	Investor time par bech kar nuksan se bach gaya
Crash nahi hoga	Crash nahi hoga	TN	Sab theek
Crash nahi hoga	Crash hoga	FP	Investor ne shares bech diye, par market nahi giri — faayda miss hua (opportunity loss), lekin paisa safe hai
Crash hoga	Crash nahi hoga	FN	Blunder! — investor ne kuch nahi kiya aur market gir gayi — badi financial loss

Aam taur par yahan **FN zyada khatarnak** hai (poora portfolio gir sakta hai), jabki FP ka nuksan limited hota hai. Isliye focus **FN kam karne par** hoga, yaani **Recall** (aur F-Beta me $\beta = 2$). Ek caveat: agar FP par bhi badi cost lagti ho (jaise bahut zyada transaction charges ya tax), toh dono ko balance karne ke liye F1 dekhna behtar hoga — *metric ka choice business cost par depend karta hai*.

15. F-Beta Score

Precision aur Recall dono ek saath achhe hona mushkil hota hai, aur hume aksar **ek hi number** chahiye jo dono ko combine kare. Yahi kaam **F-Beta Score** karta hai. Beta (β) ek dial ki tarah hai jo decide karta hai ki **Precision zyada important hai ya Recall**:

$$F_{\beta} = (1 + \beta^2) \cdot \frac{\text{Precision} \times \text{Recall}}{\beta^2 \times \text{Precision} + \text{Recall}}$$

Teen Situations

Situation	β	Formula	Kab use karein
FP aur FN dono important	1	$F1 = 2 \cdot P \cdot R / (P + R)$	Balanced cases — F1 Score (harmonic mean)
FP zyada important (FP > FN)	0.5	$F0.5 = 1.25 \cdot P \cdot R / (0.25 \cdot P + R)$	Spam classification — Precision ko zyada weight
FN zyada important (FN >> FP)	2	$F2 = 5 \cdot P \cdot R / (4 \cdot P + R)$	Disease detection, stock crash — Recall ko zyada weight

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$F_{0.5} = (1 + 0.25) \cdot \frac{P \times R}{0.25 \times P + R}$$

$$F_2 = (1 + 4) \cdot \frac{P \times R}{4 \times P + R}$$

Dhyaan dein: Teacher ne F0.5 aur F2 ke formulas me denominator me sirf (P + R) likha hai. Sahi general formula me denominator ($\beta^2 \cdot P + R$) hota hai — yaani F0.5 me $0.25 \cdot P + R$ aur F2 me $4 \cdot P + R$. F1 me $\beta = 1$ hone se $\beta^2 = 1$ aa jaata hai, isliye wahan (P + R) hi sahi hai. Numerator $(1 + \beta^2)$ wala part teacher ka bilkul sahi hai.

F1 Ko Harmonic Mean Kyun Kehte Hain?

$F1 = 2PR/(P + R)$ asal me Precision aur Recall ka **harmonic mean** hai. Arithmetic mean $(P + R)/2$ ke muqable harmonic mean **kamzor value ki taraf zyada jhukta hai**, isliye koi bhi ek metric bahut kharab ho toh F1 gir jaata hai. Example: Precision = 1.0 aur Recall = 0.0 ho toh arithmetic mean 0.5 aayega (dhokha), lekin $F1 = 0$ aayega (sahi).

Worked Example — Teacher Wale Dataset Par

Yahan Precision P = 0.60 aur Recall R = 0.75 tha:

- $F1 = 2 \times 0.60 \times 0.75 / (0.60 + 0.75) = 0.90 / 1.35 \approx \mathbf{0.667}$
- $F0.5 = 1.25 \times 0.45 / (0.25 \times 0.60 + 0.75) = 0.5625 / 0.90 = \mathbf{0.625}$ (Precision ka weight zyada, isliye chhoti Precision ki wajah se score niche)
- $F2 = 5 \times 0.45 / (4 \times 0.60 + 0.75) = 2.25 / 3.15 \approx \mathbf{0.714}$ (Recall ka weight zyada, isliye achhi Recall ki wajah se score upar)

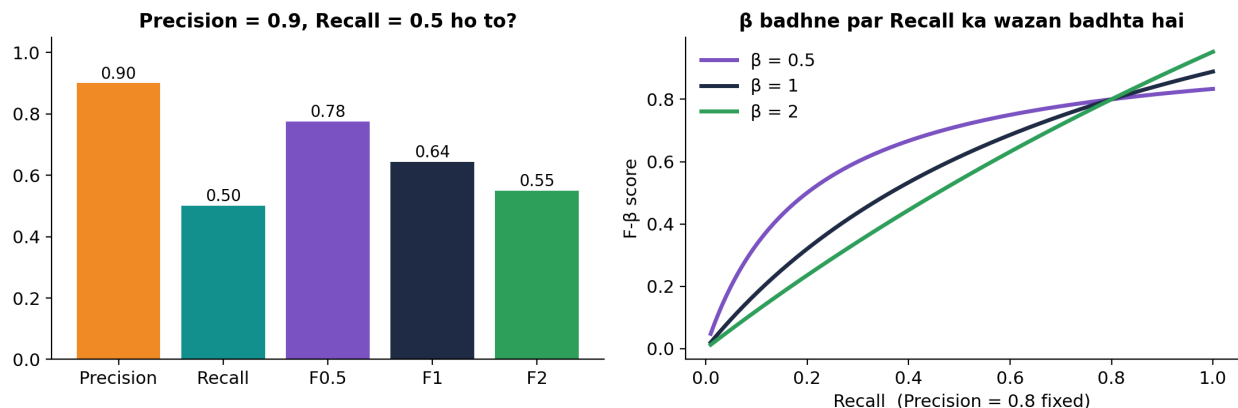


Figure 15: Left — Precision = 0.9 aur Recall = 0.5 par alag-alag β ke scores. Right — β badhne par F- β Recall ke saath zyada tezi se badhta hai.

Sab Metrics Ek Saath — Python Code

```
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, fbeta_score)

y_true = [0, 1, 0, 1, 1, 0, 1]    # actual
y_pred = [1, 1, 0, 1, 1, 1, 0]    # model ne predict kiya

print(confusion_matrix(y_true, y_pred))    # [[1 2]
                                           #  [1 3]]  -> [[TN FP],[FN TP]]
print(accuracy_score(y_true, y_pred))      # 0.571
print(precision_score(y_true, y_pred))     # 0.6
print(recall_score(y_true, y_pred))        # 0.75
print(fbeta_score(y_true, y_pred, beta=1)) # 0.667
print(fbeta_score(y_true, y_pred, beta=0.5)) # 0.625
print(fbeta_score(y_true, y_pred, beta=2)) # 0.714
```

Truth → Not diabetes
Model → Diabetes } ⇒ 2nd opinion
↓
check

Assignment

④ Tomorrow the stock market will crash or not

Reducing FPR or FNR

④ F-Beta Score : $(1 + \beta^2) \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

① If FP & FN are both important

$$\beta = 1$$

$$F1 \text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \Rightarrow \text{Harmonic Mean}$$

② If FP is more important FN

$$\beta = 0.5$$

$$F_{0.5} \text{ Score} = (1 + 0.25) \frac{P \times R}{P + R}$$

③ If FN >> FP

$$\beta = 2$$

$$F_2 \text{ Score} = (1 + 4) \frac{P \times R}{P + R}$$

Isme dikhaya gaya hai: Diabetes ka aakhri case (Truth: Not diabetes, Model: Diabetes → 2nd opinion), stock market assignment (FP ↓ ya FN ↓), aur F-Beta Score: $\beta = 1$ (F1, harmonic mean), $\beta = 0.5$ (FP zyada important), $\beta = 2$ (FN >> FP).

16. Kab Konsa Metric Chunein?

Situation	Kaun sa error zyada mehnga?	Metric	Example
Balanced classes, dono errors barabar	Dono barabar	Accuracy / F1	Simple Pass/Fail
Imbalanced data	Depends	Precision, Recall, F1 (accuracy nahi)	Fraud detection
Galat 'positive' alarm mehnga	FP	Precision / F0.5	Spam filter, recommendation
Positive miss karna mehnga	FN	Recall / F2	Disease detection, stock crash, security threat

17. Poore Notes Ka Quick Revision

Logistic Regression

- Binary classification algorithm: output 0 ya 1. Hypothesis: $\mathbf{h}_{\theta}(\mathbf{x}) = \sigma(\theta_0 + \theta_1 \mathbf{x}) = 1/(1 + e^{-z})$.
- Linear Regression classification me fail: (1) outlier se line badalti hai, (2) output > 1 ya < 0 ho sakta hai. Sigmoid output ko (0, 1) me squash karta hai.
- MSE + sigmoid = non-convex (local minima). Isliye **Log Loss**: $J = -(1/m) \sum [y \log h + (1-y) \log(1-h)]$ — convex.
- Gradient Descent: $\theta_j := \theta_j - \alpha \partial J / \partial \theta_j$; $\partial J / \partial \theta_0 = (1/m) \sum (h-y)$, $\partial J / \partial \theta_1 = (1/m) \sum (h-y)x$.
- Multi-class ke liye **OvR**: K classes $\rightarrow$ K binary classifiers (class k vs rest); prediction = **argmax** of K scores.

Performance Metrics

Metric	Formula	Ek line me
Accuracy	$(TP + TN) / (TP + FP + FN + TN)$	Kul me se kitni sahi (imbalanced data me dhokha)
Precision	$TP / (TP + FP)$	Jo positive bola, usme kitna sahi — FP kam karo
Recall	$TP / (TP + FN)$	Jitne asli positive the, unme se kitne pakde — FN kam karo
F1	$2PR / (P + R)$	Precision aur Recall ka harmonic mean ($\beta = 1$)
F- β	$(1 + \beta^2)PR / (\beta^2 P + R)$	$\beta < 1 \rightarrow$ Precision, $\beta > 1 \rightarrow$ Recall ko zyada weight

In short: Logistic Regression ek S-shaped curve (sigmoid) ke through probability nikalta hai aur usse class decide karta hai; Log Loss + Gradient Descent uske parameters seekhte hain; OvR usse multi-class tak extend karta hai; aur Confusion Matrix se nikale gaye Accuracy, Precision, Recall, F-Beta batate hain ki model business ke hisaab se kitna achha hai.